

Terceira Avaliação de Métodos Numéricos

Gustavo Bittar Gonçalves

24 de junho de 2026

1 Condução térmica bidimensional transiente

O problema físico corresponde à condução de calor em uma placa bidimensional com propriedades constantes e sem geração interna. O modelo diferencial é

$$\rho c_p \frac{\partial T}{\partial t} = k \left(\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} \right), \quad 0 < x < L_x, \quad 0 < y < L_y, \quad t > 0. \quad (1)$$

Equivalentemente,

$$\frac{\partial T}{\partial t} = \alpha \left(\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} \right), \quad \alpha = \frac{k}{\rho c_p}. \quad (2)$$

As condições de contorno gerais são de Dirichlet:

$$T(0, y, t) = T_E, \quad T(L_x, y, t) = T_D, \quad (3)$$

$$T(x, 0, t) = T_S, \quad T(x, L_y, t) = T_N, \quad (4)$$

e a condição inicial usada no domínio interno é

$$T(x, y, 0) = 300 \text{ K}. \quad (5)$$

Para o caso numérico da avaliação:

$$T(0, y, t) = 400 \text{ K}, \quad T(L_x, y, t) = 500 \text{ K}, \quad (6)$$

$$T(x, 0, t) = 600 \text{ K}, \quad T(x, L_y, t) = 200 \text{ K}. \quad (7)$$

Foram usadas as propriedades do alumínio puro

$$k = 237 \text{ W/(m K)}, \quad \rho = 2700 \text{ kg/m}^3, \quad c_p = 900 \text{ J/(kg K)},$$

logo

$$\alpha = 9,7531 \times 10^{-5} \text{ m}^2/\text{s}.$$

A placa possui $L_x = L_y = 1,0 \text{ m}$ e espessura de $5,0 \text{ mm}$. Como não há termo fonte nem perda superficial, a espessura não aparece explicitamente na equação bidimensional resolvida.

1.1 Discretização implícita

Aplicando diferenças finitas centrais de segunda ordem no espaço e avanço temporal implícito, tem-se

$$\frac{T_{i,j}^{n+1} - T_{i,j}^n}{\Delta t} = \alpha \left[\frac{T_{i+1,j}^{n+1} - 2T_{i,j}^{n+1} + T_{i-1,j}^{n+1}}{\Delta x^2} + \frac{T_{i,j+1}^{n+1} - 2T_{i,j}^{n+1} + T_{i,j-1}^{n+1}}{\Delta y^2} \right]. \quad (8)$$

Definindo

$$r_x = \frac{\alpha \Delta t}{\Delta x^2}, \quad r_y = \frac{\alpha \Delta t}{\Delta y^2},$$

obtem-se, para cada ponto interno,

$$(1 + 2r_x + 2r_y)T_{i,j}^{n+1} - r_x(T_{i+1,j}^{n+1} + T_{i-1,j}^{n+1}) - r_y(T_{i,j+1}^{n+1} + T_{i,j-1}^{n+1}) = T_{i,j}^n. \quad (9)$$

O passo de tempo foi calculado por

$$\Delta t = C \frac{\min(\Delta x, \Delta y)^2}{\alpha}. \quad (10)$$

1.2 Algoritmo de Gauss-Seidel

Isolando $T_{i,j}^{n+1}$, a iteração de Gauss-Seidel fica

$$T_{i,j}^{(m+1)} = \frac{T_{i,j}^n + r_x (T_{i-1,j}^{(m+1)} + T_{i+1,j}^{(m)}) + r_y (T_{i,j-1}^{(m+1)} + T_{i,j+1}^{(m)})}{1 + 2r_x + 2r_y}. \quad (11)$$

No programa foi usada a forma Gauss-Seidel vermelho-preto vetorizada, que preserva a dependência atualizada entre vizinhos e reduz o custo computacional em Python.

1. Inicializar T^0 com 300 K no interior e impor as temperaturas prescritas no contorno.
2. Para cada passo de tempo, calcular r_x e r_y .
3. Repetir Gauss-Seidel até que $\max |T^{(m+1)} - T^{(m)}| < 10^{-8}$.
4. Atualizar o campo temporal e salvar os instantes pedidos.

Tabela 1: Parâmetros efetivos da simulação com $N_x = N_y = 10$.

C	Δt (s)	r_x	r_y	iterações médias GS
0,25	31,578947	0,249474	0,249474	7,03
0,50	63,157895	0,498947	0,498947	10,92
1,00	126,315790	0,997895	0,997895	18,08
2,00	252,631580	1,995790	1,995790	30,82

1.3 Campos de temperatura

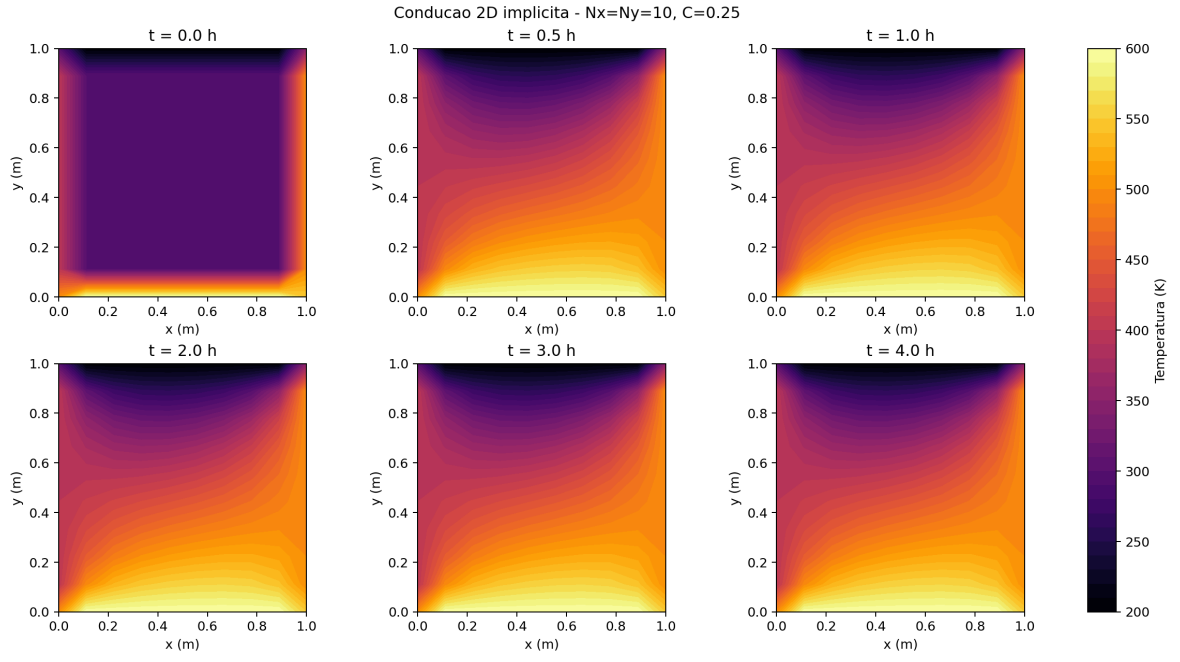


Figura 1: Campos de temperatura para $C = 0,25$.

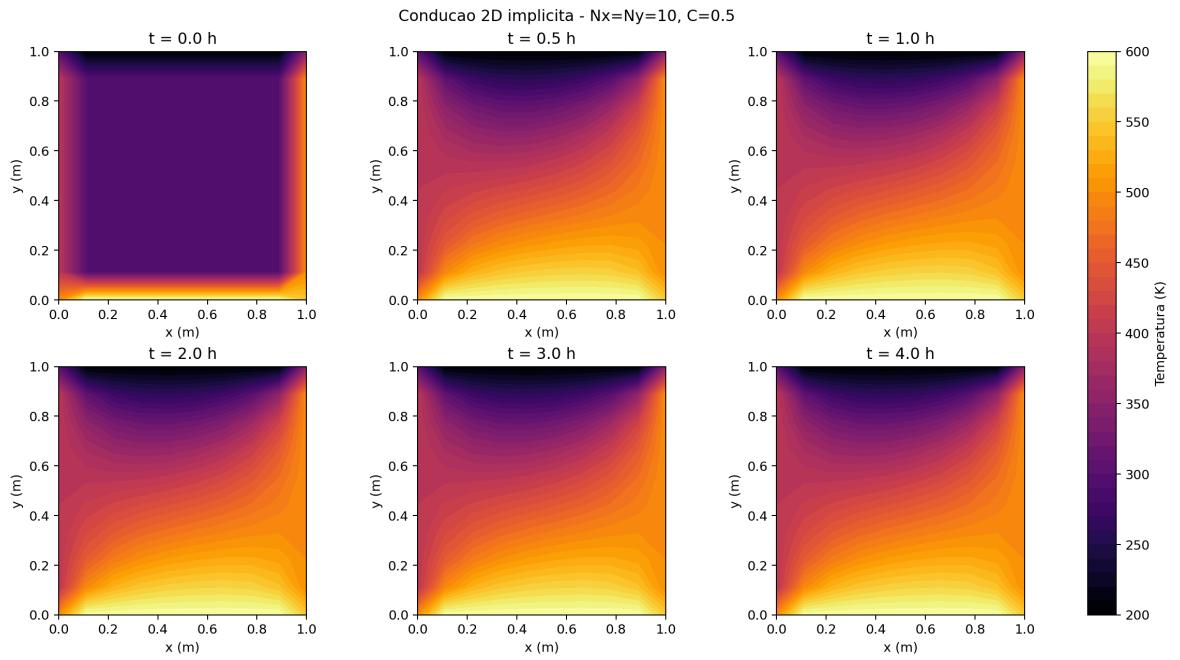


Figura 2: Campos de temperatura para $C = 0,50$.

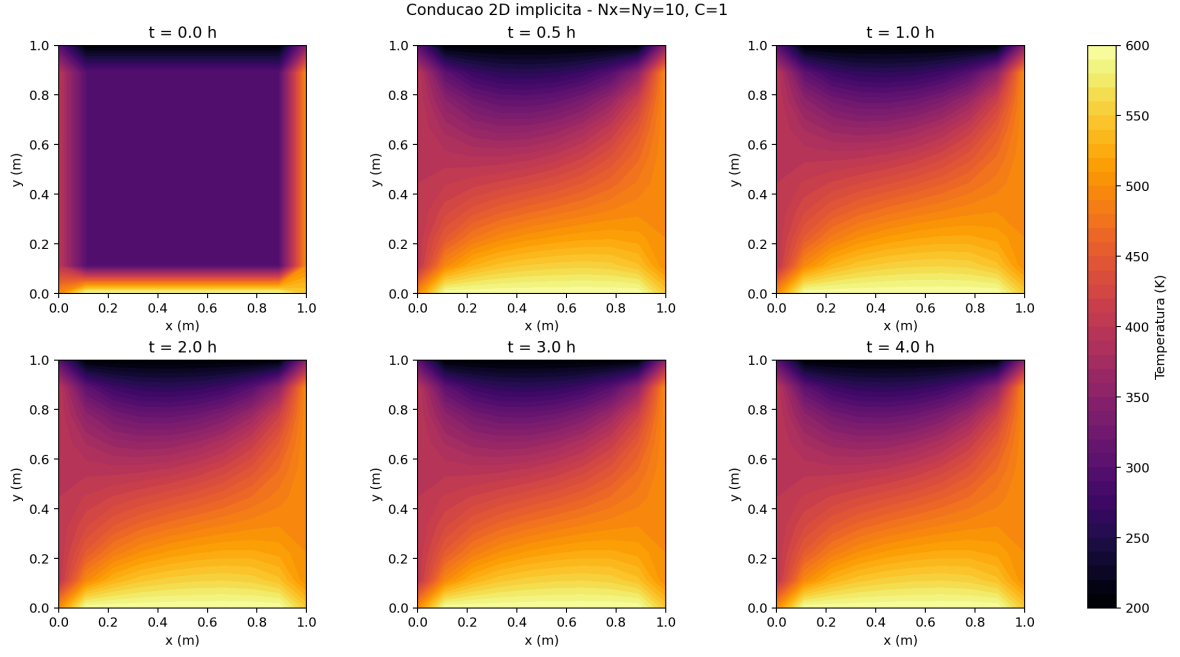


Figura 3: Campos de temperatura para $C = 1,0$.

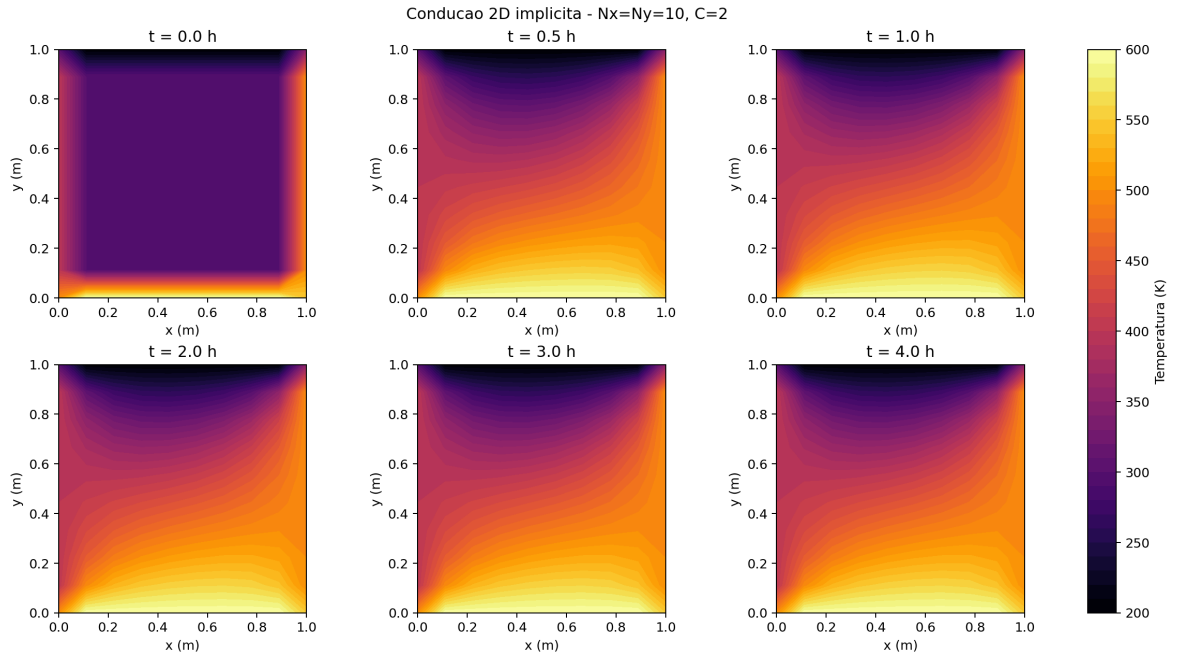


Figura 4: Campos de temperatura para $C = 2,0$.

1.4 Perfis em $t = 4$ h

Para os perfis de malha foi usado $C = 1,0$, mantendo o método implícito e comparando $N_x = N_y = 10, 20$ e 40 .

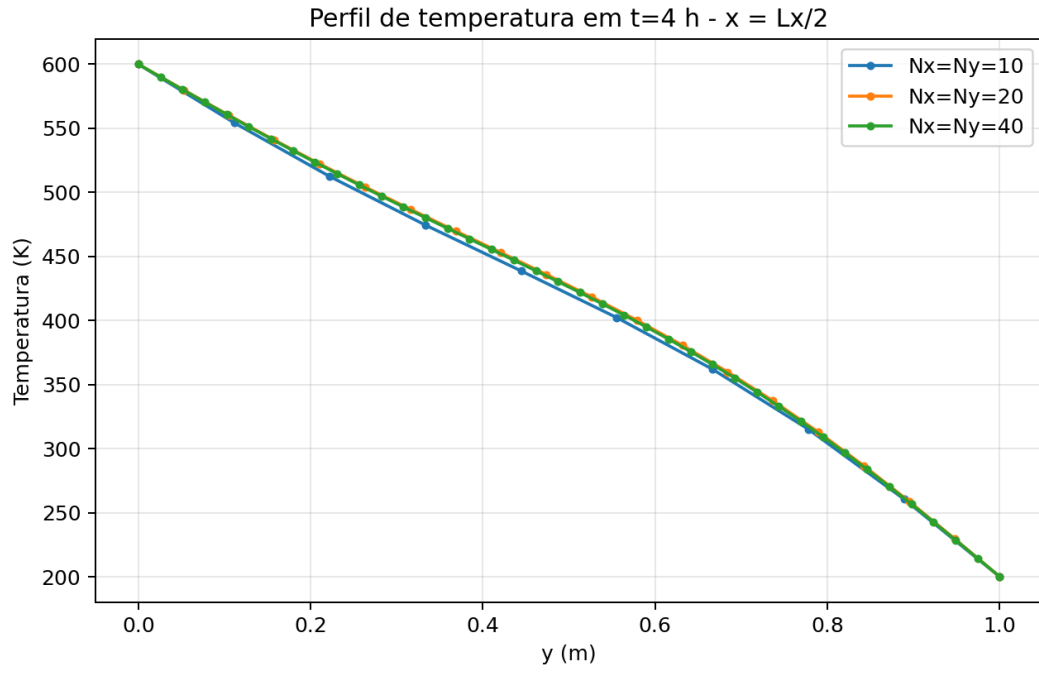


Figura 5: Perfil em $x = L_x/2$.

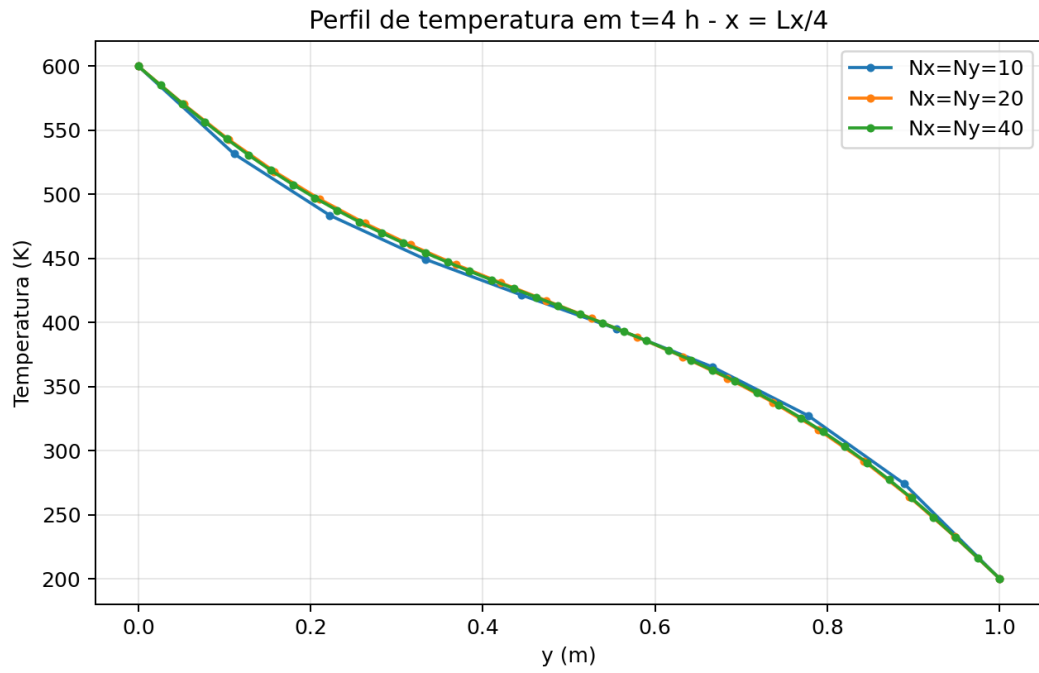


Figura 6: Perfil em $x = L_x/4$.

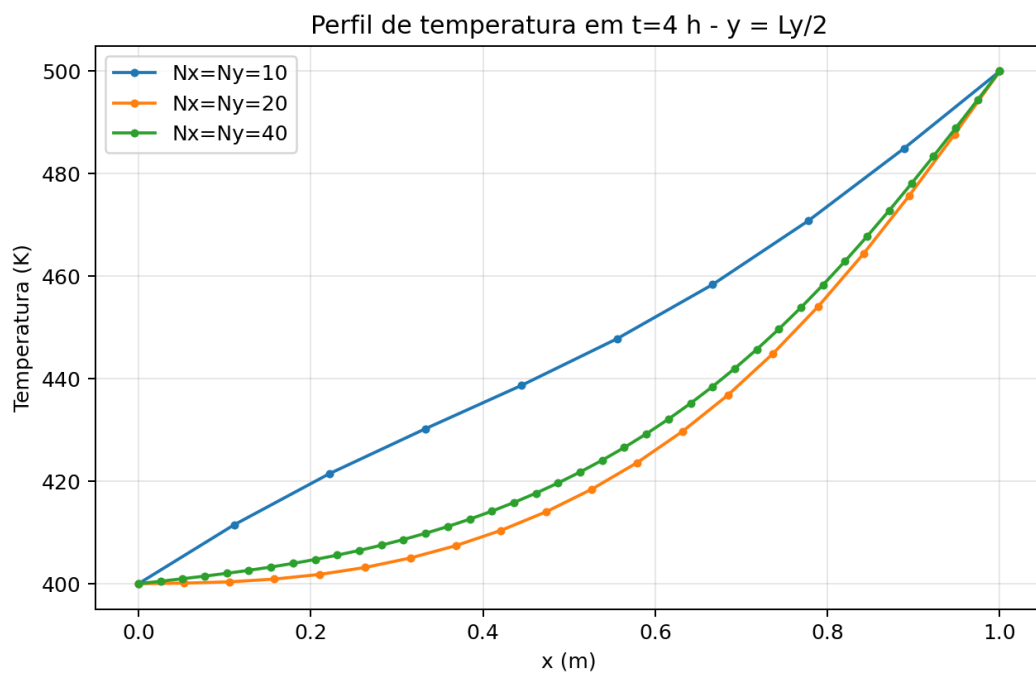


Figura 7: Perfil em $y = L_y/2$.

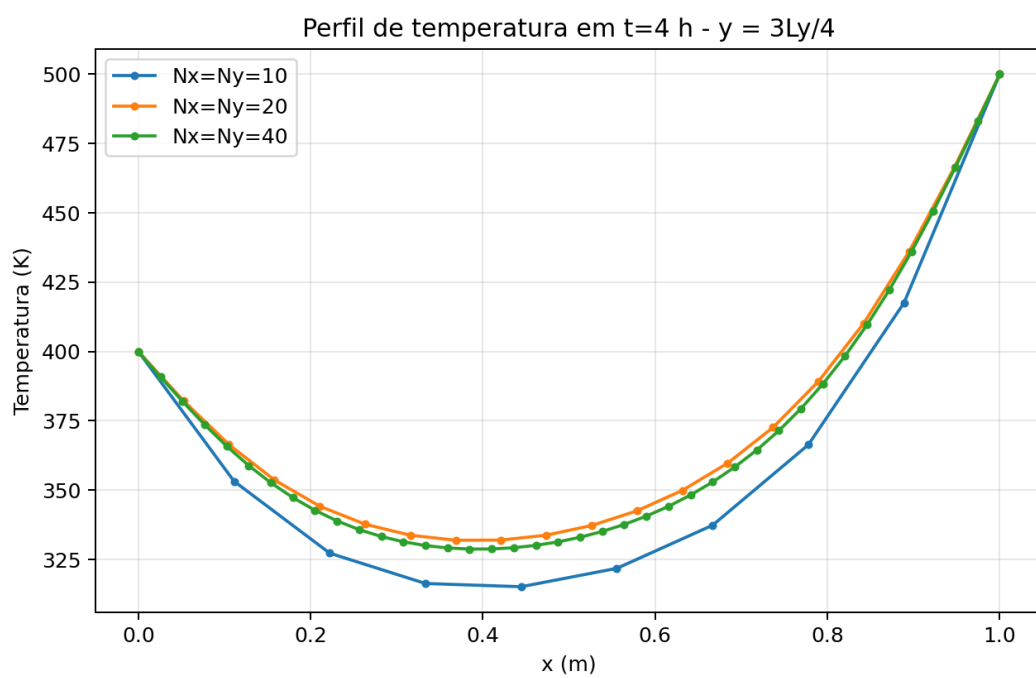


Figura 8: Perfil em $y = 3L_y/4$.

1.5 Evolução temporal

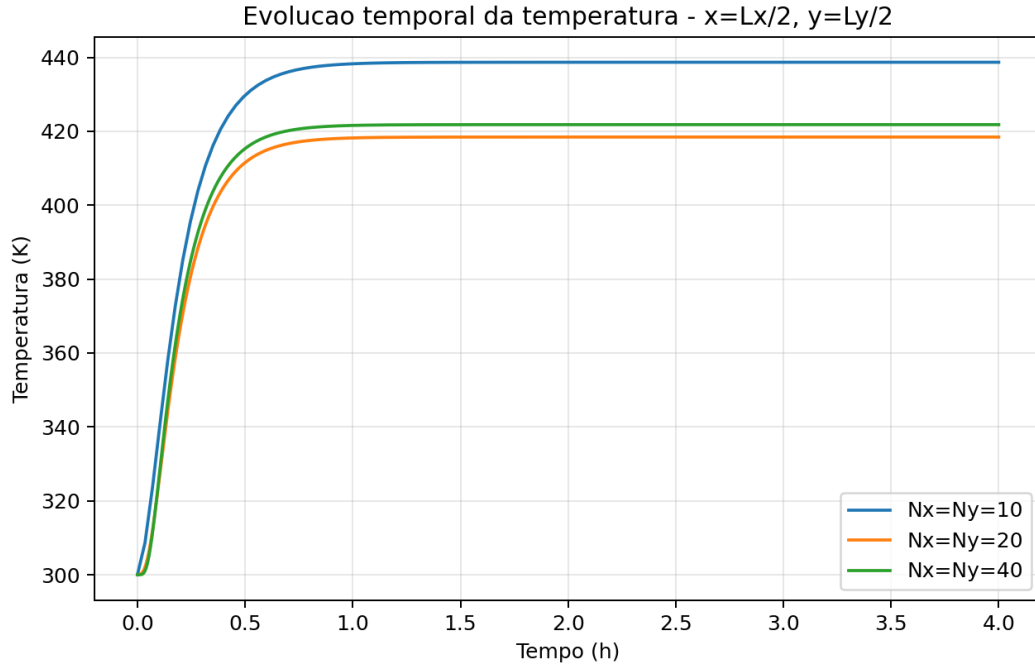


Figura 9: Evolução temporal da temperatura em $x = L_x/2$ e $y = L_y/2$.

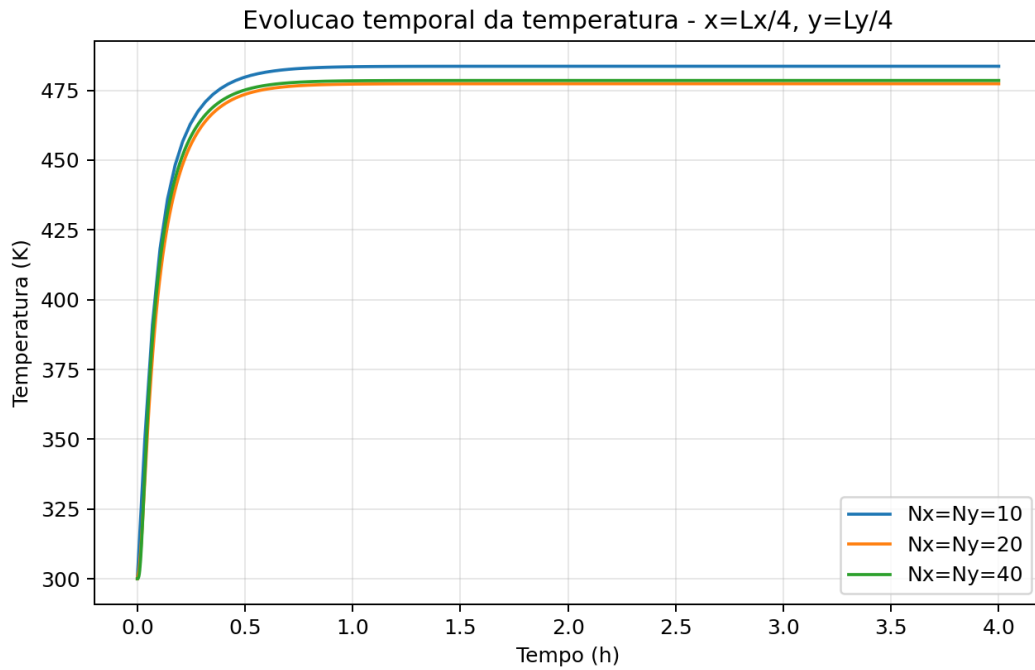


Figura 10: Evolução temporal da temperatura em $x = L_x/4$ e $y = L_y/4$.

2 Equação da onda unidimensional

A equação da onda é

$$\frac{\partial^2 y}{\partial t^2} = c^2 \frac{\partial^2 y}{\partial x^2}, \quad 0 \leq x \leq L, \quad 0 \leq t \leq 2 \text{ s.} \quad (12)$$

Foram usados $L = 1$ m, $c = 1$ m/s e $A = 0,01$ m, com

$$y(0, t) = y(L, t) = 0, \quad y(x, 0) = A \sin\left(\frac{\pi x}{L}\right), \quad v(x, 0) = \frac{\partial y}{\partial t}(x, 0) = 0. \quad (13)$$

Definindo $v = \partial y / \partial t$, o sistema de primeira ordem é

$$\frac{\partial y}{\partial t} = v, \quad \frac{\partial v}{\partial t} = c^2 \frac{\partial^2 y}{\partial x^2}. \quad (14)$$

No espaço,

$$\frac{dy_i}{dt} = v_i, \quad \frac{dv_i}{dt} = c^2 \frac{y_{i+1} - 2y_i + y_{i-1}}{\Delta x^2}. \quad (15)$$

2.1 Euler explícito

$$y_i^{n+1} = y_i^n + \Delta t v_i^n, \quad (16)$$

$$v_i^{n+1} = v_i^n + \Delta t c^2 \frac{y_{i+1}^n - 2y_i^n + y_{i-1}^n}{\Delta x^2}. \quad (17)$$

Esse método, aplicado diretamente ao sistema de primeira ordem da onda, é instável para o oscilador sem amortecimento. Por isso os resultados de erro crescem rapidamente.

2.2 Runge-Kutta explícito de quarta ordem

Se $\mathbf{u} = [\mathbf{y}, \mathbf{v}]^T$ e $\mathbf{u}' = F(\mathbf{u})$, então

$$\mathbf{k}_1 = F(\mathbf{u}^n), \quad (18)$$

$$\mathbf{k}_2 = F(\mathbf{u}^n + \frac{\Delta t}{2} \mathbf{k}_1), \quad (19)$$

$$\mathbf{k}_3 = F(\mathbf{u}^n + \frac{\Delta t}{2} \mathbf{k}_2), \quad (20)$$

$$\mathbf{k}_4 = F(\mathbf{u}^n + \Delta t \mathbf{k}_3), \quad (21)$$

$$\mathbf{u}^{n+1} = \mathbf{u}^n + \frac{\Delta t}{6} (\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4). \quad (22)$$

2.3 Euler implícito

Com $\mathbf{D}_{xx}$ representando a matriz de segunda derivada nos pontos internos,

$$\mathbf{y}^{n+1} - \Delta t \mathbf{v}^{n+1} = \mathbf{y}^n, \quad (23)$$

$$-\Delta t c^2 \mathbf{D}_{xx} \mathbf{y}^{n+1} + \mathbf{v}^{n+1} = \mathbf{v}^n. \quad (24)$$

Logo, o sistema linear resolvido a cada passo é

$$\begin{bmatrix} \mathbf{I} & -\Delta t \mathbf{I} \\ -\Delta t c^2 \mathbf{D}_{xx} & \mathbf{I} \end{bmatrix} \begin{bmatrix} \mathbf{y}^{n+1} \\ \mathbf{v}^{n+1} \end{bmatrix} = \begin{bmatrix} \mathbf{y}^n \\ \mathbf{v}^n \end{bmatrix}. \quad (25)$$

A matriz $\mathbf{D}_{xx}$ é tridiagonal:

$$\mathbf{D}_{xx} = \frac{1}{\Delta x^2} \begin{bmatrix} -2 & 1 & 0 & \cdots & 0 \\ 1 & -2 & 1 & \cdots & 0 \\ 0 & 1 & -2 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & 1 & -2 \end{bmatrix}. \quad (26)$$

2.4 Solução analítica e erro

A solução analítica usada para a comparação é

$$y(x, t) = A \sin\left(\frac{\pi x}{L}\right) \cos\left(\frac{\pi ct}{L}\right). \quad (27)$$

O erro discreto foi calculado por

$$E(t) = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i^{\text{num}} - y_i^{\text{exata}})^2}. \quad (28)$$

2.5 Resultados da onda

As simulações usaram $N_x = 101$ e $\Delta t = 0,004$ s.

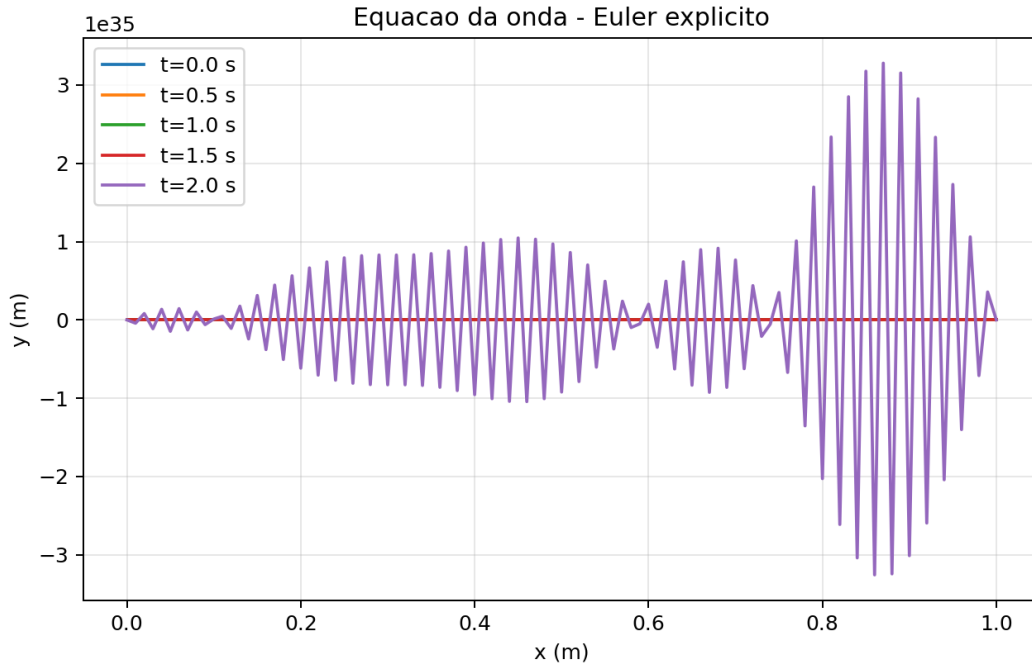


Figura 11: Deslocamento $y(x, t)$ por Euler explícito.

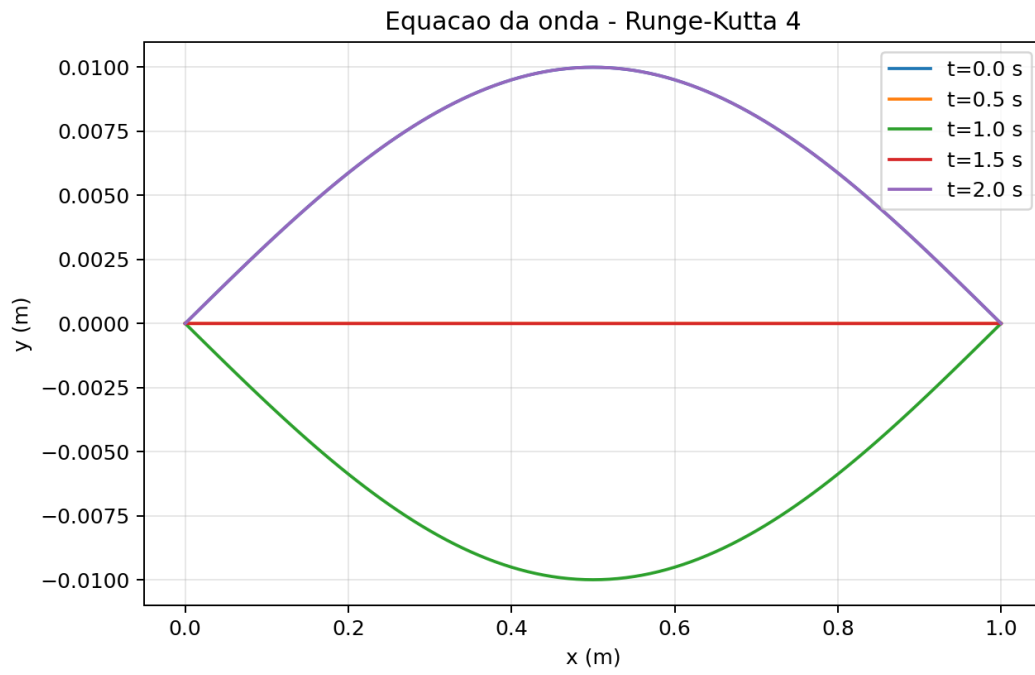


Figura 12: Deslocamento $y(x, t)$ por Runge-Kutta de quarta ordem.

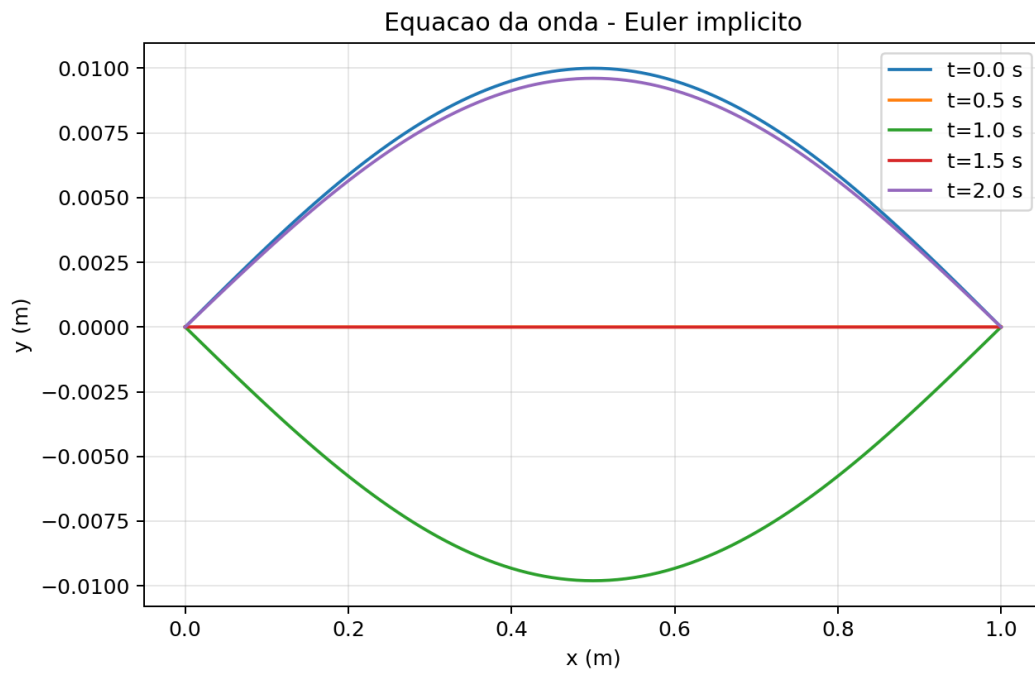


Figura 13: Deslocamento $y(x, t)$ por Euler implícito.

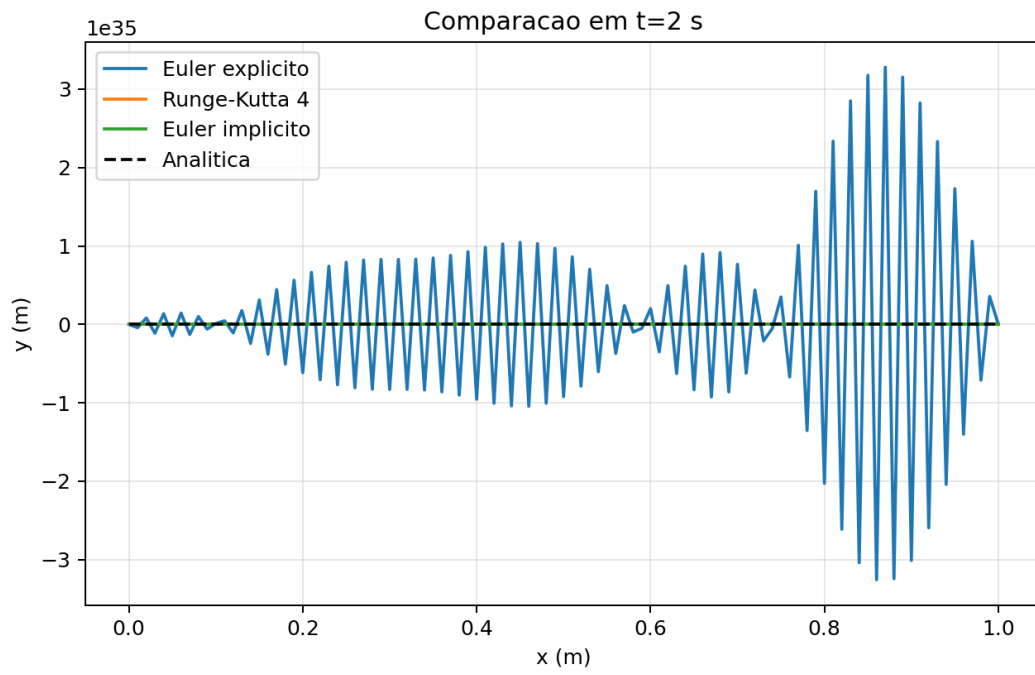


Figura 14: Comparação dos métodos em $t = 2$ s.

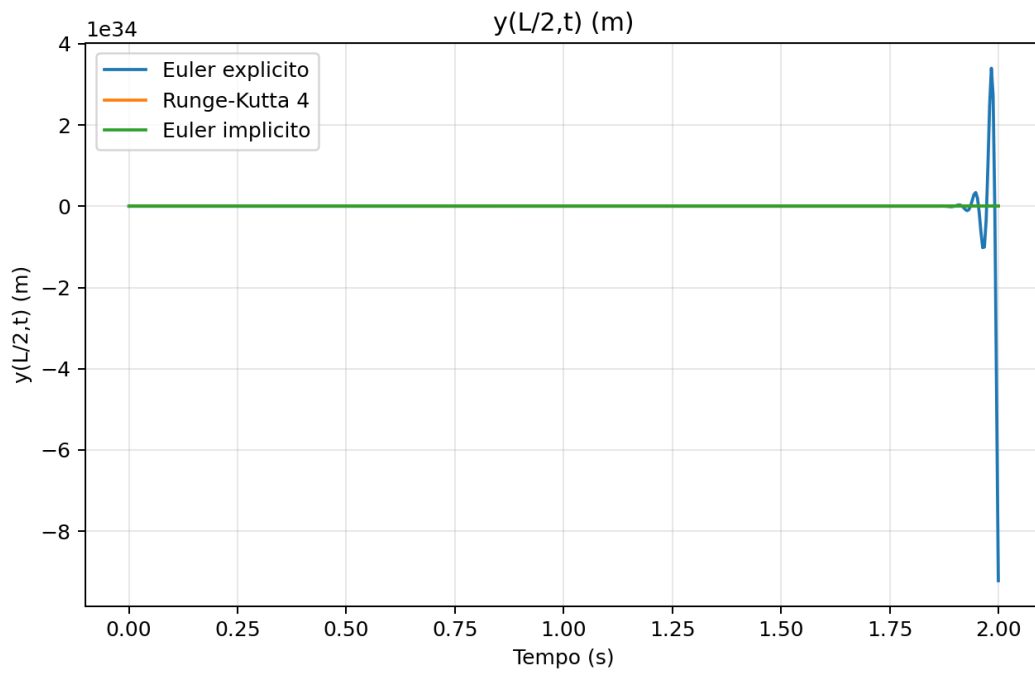


Figura 15: Evolução temporal de $y(L/2, t)$.

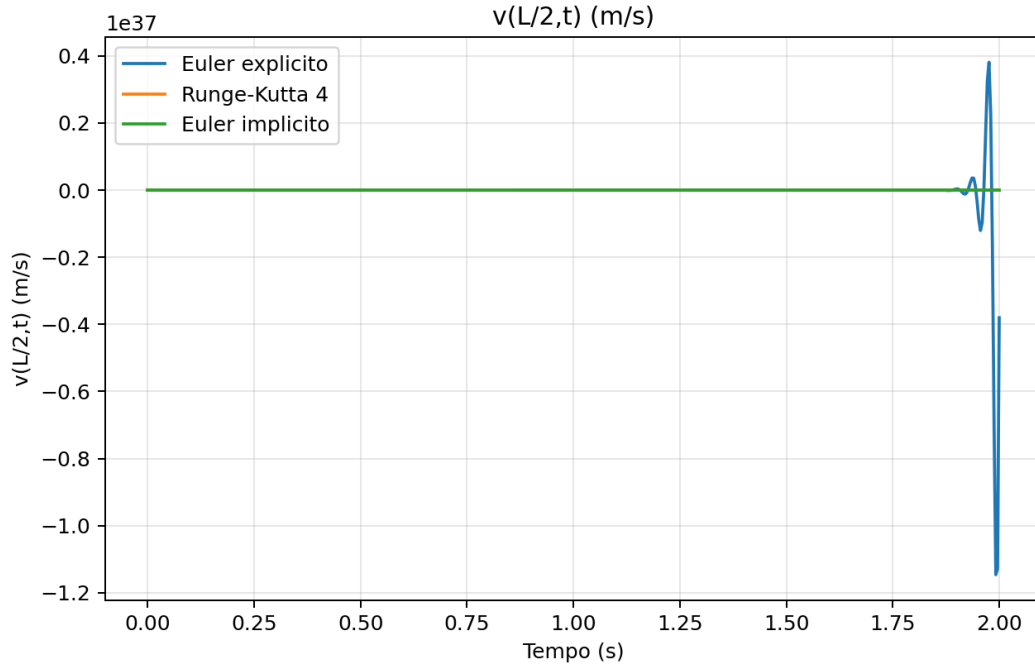


Figura 16: Evolução temporal de $v(L/2, t)$.

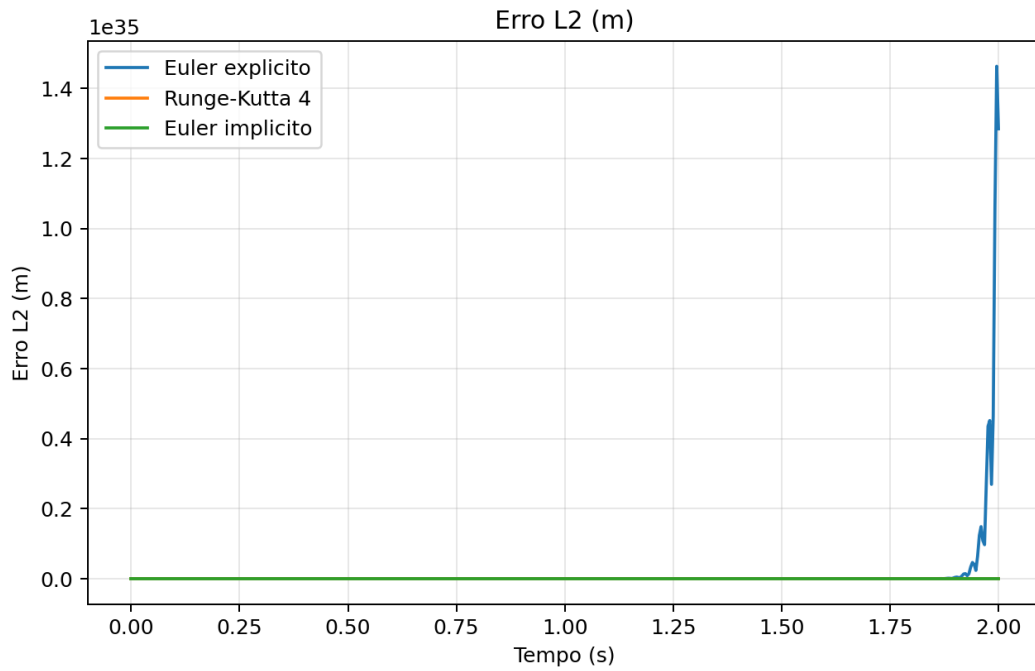


Figura 17: Evolução temporal do erro L_2 .

3 Programas computacionais

O código completo está no arquivo `main.py`. Ele executa todas as simulações e grava automaticamente as figuras usadas neste relatório. Para atender ao pedido de três programas computacionais da questão 9, também foram criados arquivos de execução separados para

cada método da onda:

`onda_euler_explicito.py`, `onda_rk4.py`, `onda_euler_implicito.py`.

Cada um desses arquivos chama a mesma implementação numérica validada em `main.py`, evitando divergência entre o relatório e os programas.

```
1 def run_all() -> None:
2     ensure_dirs()
3     heat_cfg = HeatConfig()
4     heat_times = [0.0, 1800.0, 3600.0, 7200.0, 10800.0, 14400.0]
5     heat_c_results = {}
6     for c_value in [0.25, 0.50, 1.0, 2.0]:
7         result = simulate_heat(10, 10, c_value, heat_times,
8                                 heat_cfg)
9         heat_c_results[c_value] = result
10        plot_heat_snapshots(result, c_value)
11
12    heat_mesh_results = {}
13    for n in [10, 20, 40]:
14        heat_mesh_results[n] = simulate_heat(n, n, 1.0, [0.0,
15        14400.0], heat_cfg)
16    plot_heat_mesh_profiles(heat_mesh_results, heat_cfg)
17    plot_heat_temporal(heat_mesh_results)
18
19    wave_cfg = WaveConfig()
20    wave_results = {
21        "euler_explicito": simulate_wave("euler_explicito",
22        wave_cfg),
23        "rk4": simulate_wave("rk4", wave_cfg),
24        "euler_implicito": simulate_wave("euler_implicito",
25        wave_cfg),
26    }
27    plot_wave_results(wave_results, wave_cfg)
28    save_summary(heat_c_results, wave_results)
```

4 Conclusões

O método implícito para condução térmica produziu campos suaves e convergentes para todos os coeficientes C testados. O aumento de C elevou o número médio de iterações de Gauss-Seidel por passo, como esperado, pois o acoplamento espacial do sistema linear se torna mais forte.

Na equação da onda, o método RK4 acompanhou muito bem a solução analítica no intervalo simulado. O Euler implícito permaneceu estável, mas introduziu amortecimento numérico. O Euler explícito aplicado diretamente ao sistema de primeira ordem apresentou crescimento instável, refletido no erro L_2 .

5 Referência

INCROPERA, F. P.; DEWITT, D. P.; BERGMAN, T. L.; LAVINE, A. S. *Fundamentos de transferência de calor e de massa*. LTC.